🔑 File Handling এর বেসিক স্টেপ

Python এ file handle করার জন্য আমরা মূলত `open()` ফাংশন ব্যবহার করি।

```
open(filename, mode)
```

Mode গুলো:

- `"r"` → read (ডিফল্ট, শুধু পড়ার জন্য)
- `"w"` → write (নতুন লিখবে, আগের কনটেন্ট মুছে ফেলবে)
- `"a"` → append (আগের কনটেন্ট রেখে শেষে নতুন যোগ করবে)
- `"x"` → create (নতুন ফাইল বানাবে, ফাইল থাকলে error দিবে)
- `"b"` → binary mode (image, video এর জন্য)
- `"t"` → text mode (ডিফল্ট)

১) ফাইল ওপেন ও রিড করা

```
# sample.txt এর ভেতরে কিছু লেখা আছে ধরে নিচ্ছি
f = open("sample.txt", "r")   # read mode
content = f.read()
print(content)
f.close()
```

২) এক লাইন করে পড়া

```
f = open("sample.txt", "r")
for line in f:
    print(line.strip())
f.close()
```

৩) ফাইলে লেখা

```
f = open("sample.txt", "w")   # আগের কনটেন্ট মুছে ফেলবে
f.write("Hello, I am learning Python File Handling!\n")
f.close()
```

৪) ফাইলে নতুন যোগ করা (append)

```
f = open("sample.txt", "a")
f.write("This is an extra line.\n")
f.close()
```

৫) with স্টেটমেন্ট (Best Practice ✅)

এভাবে লিখলে আলাদা করে `close()` দিতে হয় না:

```
with open("sample.txt", "r") as f:
    data = f.read()
    print(data)
```

৬) Binary ফাইল (image/video পড়া/লেখা)

```
# Image পড়া
with open("image.jpg", "rb") as f:
    data = f.read()
```

🔎 এখানে কী হচ্ছে?

- `"rb"` মানে হলো → ফাইলকে **read binary mode** এ খোলা।
- `f.read()` → পুরো ইমেজ ফাইলের **raw binary data** (bytes) পড়া হবে।

অর্থাৎ `data` ভেরিয়েবল আসলে একটি **bytes object** হবে।

✅ Output কেমন হবে?

যদি তুমি `print(data)` চালাও, তাহলে অনেক লম্বা কিছু এভাবে দেখা যাবে (random উদাহরণ):

```
b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x02\x00\x00\x01\x00\x01\x00\x00...'
```

- `b''` মানে হচ্ছে এটা **bytes data**
- ভেতরের অংশে অনেকগুলো `\x..` ফরম্যাটে হেক্স ভ্যালু থাকবে।
- এগুলো আসলে ইমেজ ফাইলের raw byte content।

⚠ তাই মনে রেখো:

- Binary data মানুষের পড়ার মতো নয়।
- এই ডেটা আবার `wb` mode এ লিখলে আসল ছবিটা ফিরে আসবে:

```
with open("copy.jpg", "wb") as f:
    f.write(data)
```

এবার `copy.jpg` খুললে আসল ইমেজ দেখা যাবে।

👉 তাহলে সংক্ষেপে: **আউটপুট হবে bytes আকারে ইমেজ ফাইলের raw ডেটা**।

৭) সাধারণ Example: File Copy

```
with open("sample.txt", "r") as src:
    content = src.read()

with open("copy.txt", "w") as dest:
    dest.write(content)

print("File copied successfully!")
```

Python দিয়ে image file আউটপুটে দেখাতে চাইলে দুইভাবে করা যায়:

▯ ১) Jupyter Notebook / Google Colab এ

তুমি যদি Jupyter বা Colab ব্যবহার করো, তাহলে সহজেই ছবি দেখানো যাবে:

```
from PIL import Image
from IPython.display import display

# ইমেজ ওপেন করো
img = Image.open("Nusrat PP.jpg")

# আউটপুটে দেখাও
display(img)
```

☞ এটা করলে ছবিটা নোটবুক আউটপুটে সরাসরি দেখাবে।

▯ ২) Python script (লোকাল কম্পিউটারে)

যদি তুমি সাধারণ `.py` ফাইল চালাও, তাহলে ছবি একটা **window** তে খুলে যাবে:

```
from PIL import Image

img = Image.open("Nusrat PP.jpg")
img.show()
```

☞ এখানে `img.show()` ছবিটাকে ডিফল্ট image viewer দিয়ে খুলে দেবে।

▯ ৩) (অতিরিক্ত) Matplotlib দিয়ে দেখানো

আরও একটা সাধারণ উপায় হলো `matplotlib` ব্যবহার করা:

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

img = mpimg.imread("Nusrat PP.jpg")
plt.imshow(img)
plt.axis("off")   # axis সরানোর জন্য
plt.show()
```

✨ সংক্ষেপে:

- **Notebook/Colab** → `display(img)`
- **Local Python Script** → `img.show()`
- **Visualization/Customization দরকার হলে** → `matplotlib`

Working with PDF:

1) PDF পড়া (Extract text)

Python-এ জনপ্রিয় লাইব্রেরি:

- `PyPDF2` → simple reading & writing
- `pdfplumber` → accurate text extraction
- `fitz` (PyMuPDF) → text + images + pages

◆ PyPDF2 Example (text read)

```
import PyPDF2

file_path = "sample.pdf"

# Open PDF in read-binary mode
with open(file_path, "rb") as f:
    reader = PyPDF2.PdfReader(f)
    print("Total pages:", len(reader.pages))

    # Read all pages
    for i, page in enumerate(reader.pages):
        text = page.extract_text()
        print(f"--- Page {i+1} ---")
        print(text)
```

---

2) PDF থেকে specific page extract করা

```
import PyPDF2

with open("sample.pdf", "rb") as f:
    reader = PyPDF2.PdfReader(f)
    writer = PyPDF2.PdfWriter()

    # Only page 0 (first page)
    writer.add_page(reader.pages[0])

    with open("first_page.pdf", "wb") as out:
        writer.write(out)
print("First page saved as first_page.pdf")
```

---

3) PDF merge (multiple PDFs একসাথে)

```
import PyPDF2

pdfs = ["file1.pdf", "file2.pdf"]
writer = PyPDF2.PdfWriter()

for pdf in pdfs:
    reader = PyPDF2.PdfReader(pdf)
    for page in reader.pages:
        writer.add_page(page)

with open("merged.pdf", "wb") as out:
    writer.write(out)
print("PDFs merged successfully")
```

---

4) PDF split (page by page)

```
import PyPDF2

with open("sample.pdf", "rb") as f:
    reader = PyPDF2.PdfReader(f)
    for i, page in enumerate(reader.pages):
        writer = PyPDF2.PdfWriter()
        writer.add_page(page)
        with open(f"page_{i+1}.pdf", "wb") as out:
            writer.write(out)
print("PDF split into separate pages")
```

---

5) Advanced: Images / OCR

- Text extract করতে পারলে ভালো, কিন্তু scanned PDF হলে `PyPDF2` text extract করতে পারবে না।
- সেই ক্ষেত্রে `pdfplumber` বা `pytesseract` (OCR) ব্যবহার করতে হবে।

```
# pdfplumber example
import pdfplumber

with pdfplumber.open("scanned.pdf") as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        print(text)
```

✅ সংক্ষেপে:

- **Read text** → PyPDF2 / pdfplumber
- **Write / Merge / Split** → PyPDF2
- **Scanned PDF OCR** → pytesseract + pdfplumber

2. Text Extract (PyPDF2 দিয়ে)

```
from PyPDF2 import PdfReader

reader = PdfReader("file.pdf")
for page in reader.pages:
    text = page.extract_text()
    print(text)
```

👉 এভাবে PDF এর ভেতরের আসল টেক্সট পড়া যায়।

📝 **Summary**

- `%PDF-1.5` মানে হচ্ছে → এটা একটা **PDF ফাইল**।
- তোমার কোড ভুলে এটাকে **text file হিসেবে** ওপেন করেছে।
- সঠিকভাবে পড়তে হলে **binary mode (`rb`)** বা **PyPDF2**/`pdfplumber` এর মতো লাইব্রেরি ব্যবহার করতে হবে।

Python Generator:

1. Generator কী?

👉 Python-এ **generator** হলো এমন এক ধরনের function/iterator যেটা একবারে সব ডেটা মেমোরিতে না এনে **একটা একটা করে ডেটা তৈরি করে দেয়**।

- Normal function সবকিছু একবারে return করে।
- Generator function বারবার `yield` দিয়ে value return করে।

---

2. কেন Generator ব্যবহার করব?

- **Memory efficient** → একসাথে পুরো list বানাতে হবে না।
- **Lazy evaluation** → প্রয়োজন অনুযায়ী মান তৈরি হয়।
- **Infinite sequence তৈরি করা যায়** → যেমন ফিবোনাচ্চি সিরিজ, স্ট্রিমিং ডেটা ইত্যাদি।

---

3. Generator Function (using `yield`)

```
def my_generator():
    print("First yield")
    yield 1   # প্রথম মান ফেরত দেবে

    print("Second yield")
    yield 2   # দ্বিতীয় মান ফেরত দেবে

    print("Third yield")
    yield 3   # তৃতীয় মান ফেরত দেবে

# জেনারেটর কল করলে শুধু iterator তৈরি হয়
gen = my_generator()

print(next(gen))  # 1
print(next(gen))  # 2
print(next(gen))  # 3
```

⚡ এখানে লক্ষ্য করো → function একবারে শেষ হচ্ছে না, বরং প্রতিবার `yield` থেকে pause হচ্ছে এবং পরেরবার আবার resume হচ্ছে।

---

4. Generator Expression

List comprehension-এর মতো syntax আছে, কিন্তু `[]` এর বদলে `()` ব্যবহার করতে হয়।

```
# Normal list comprehension
squares = [x*x for x in range(5)]
print(squares)   # [0, 1, 4, 9, 16]

# Generator expression
squares_gen = (x*x for x in range(5))
print(squares_gen)      # শুধু iterator object
print(next(squares_gen))  # 0
print(next(squares_gen))  # 1
```

---

5. Practical Example: বড় ফাইল line by line পড়া

```
def read_file(filename):
    with open(filename, "r") as f:
        for line in f:
            yield line.strip()

for line in read_file("bigdata.txt"):
    print(line)
```

👉 এভাবে বড় ফাইল মেমোরিতে পুরোটা লোড না করে **এক লাইন করে পড়তে পারবে**।

---

6. Infinite Generator Example

```
def infinite_numbers():
    n = 1
    while True:
        yield n
        n += 1

gen = infinite_numbers()
print(next(gen))  # 1
print(next(gen))  # 2
print(next(gen))  # 3
```

Summary:

- `yield` দিয়ে generator বানানো যায়।
- একবারে সবকিছু না বানিয়ে একে একে value তৈরি করে।
- Memory efficient, infinite sequence সম্ভব।
- Generator expression ব্যবহার করে ছোট করে লেখা যায়।